

C programming has three types of loops:

1. for loop
2. while loop
3. do...while loop

for Loop

The syntax of the for loop is:

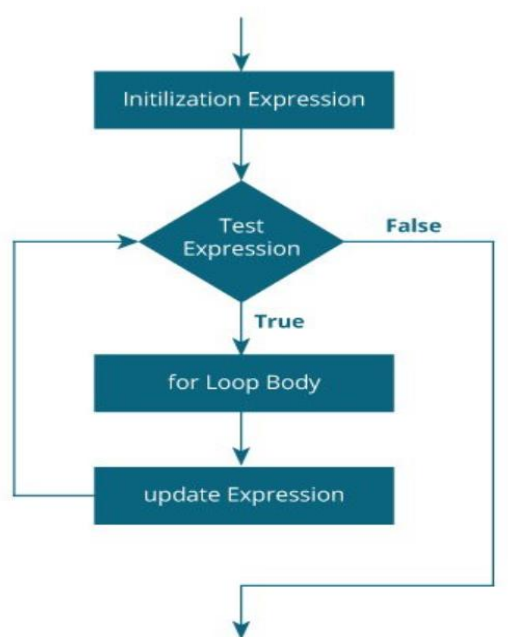
```
for (initializationStatement; testExpression; updateStatement)
{
    // statements inside the body of loop
}
```

How for loop works?

- The initialization statement is executed only once.
- Then, the test expression is evaluated. If the test expression is evaluated to false, the for loop is terminated.
- However, if the test expression is evaluated to true, statements inside the body of the for loop are executed, and the update expression is updated.
- Again the test expression is evaluated.

This process goes on until the test expression is false. When the test expression is false, the loop terminates.

for loop Flowchart



Example 1: for loop

```
// Print numbers from 1 to 10
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i < 11; ++i)
    {
        printf("%d ", i);
    }
    return 0;
}
```

Output

1 2 3 4 5 6 7 8 9 10

1. i is initialized to 1.
2. The test expression $i < 11$ is evaluated. Since 1 less than 11 is true, the body of for loop is executed. This will print the **1** (value of i) on the screen.
3. The update statement $++i$ is executed. Now, the value of i will be 2. Again, the test expression is evaluated to true, and the body of for loop is executed. This will print **2** (value of i) on the screen.
4. Again, the update statement $++i$ is executed and the test expression $i < 11$ is evaluated. This process goes on until i becomes 11.
5. When i becomes 11, $i < 11$ will be false, and the for loop terminates.

while loop

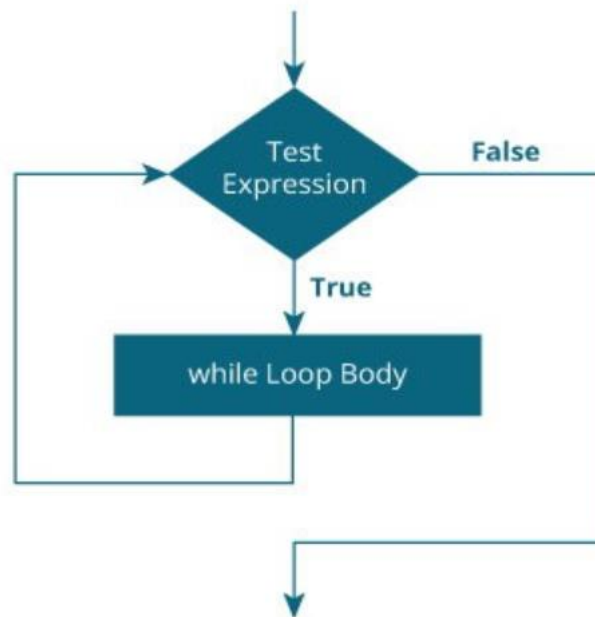
The syntax of the while loop is:

```
while (testExpression) {
    // the body of the loop
}
```

How while loop works?

- The while loop evaluates the testExpression inside the parentheses ().
- If testExpression is **true**, statements inside the body of while loop are executed. Then, testExpression is evaluated again.
- The process goes on until testExpression is evaluated to **false**.
- If testExpression is **false**, the loop terminates (ends).

Flowchart of while loop



Example: while loop

```
// Print numbers from 1 to 5
#include <stdio.h>
int main() {
    int i = 1;
    while (i <= 5) {
        printf("%d\n", i);
        ++i;
    }
    return 0;
}
```

Output

1
2
3
4
5

- i). Here, we have initialized `i` to 1.
- ii). When `i = 1`, the test expression `i <= 5` is **true**. Hence, the body of the while loop is executed. This prints 1 on the screen and the value of `i` is increased to 2.
- iii). Now, `i = 2`, the test expression `i <= 5` is again **true**. The body of the while loop is executed again. This prints 2 on the screen and the value of `i` is increased to 3.

- iv). This process goes on until `i` becomes 6. Then, the test expression `i <= 5` will be **false** and the loop terminates.

do...while loop

The do..while loop is similar to the while loop with one important difference. The body of do...while loop is executed at least once. Only then, the test expression is evaluated.

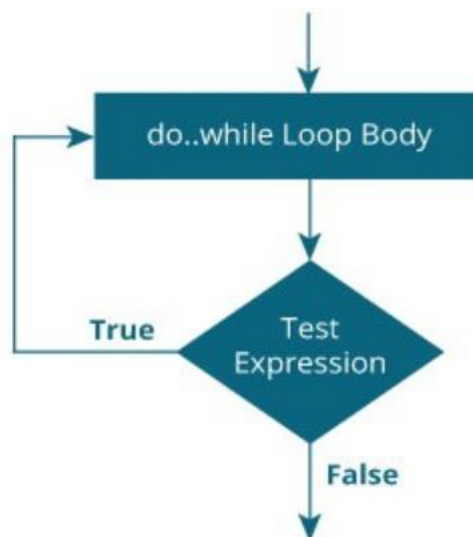
The syntax of the do...while loop is:

```
do {  
    // the body of the loop  
}  
while (testExpression);
```

How do...while loop works?

- The body of do...while loop is executed once. Only then, the testExpression is evaluated.
- If testExpression is **true**, the body of the loop is executed again and testExpression is evaluated once more.
- This process goes on until testExpression becomes **false**.
- If testExpression is **false**, the loop ends.

Flowchart of do...while Loop



Example: do...while loop

```
// Program to add numbers until the user enters zero  
#include <stdio.h>  
int main() {  
    double number, sum = 0;  
    // the body of the loop is executed at least once
```

```
do {  
    printf("Enter a number: ");  
    scanf("%lf", &number);  
    sum += number;  
}  
while(number != 0.0);  
printf("Sum = %.2lf",sum);  
return 0;  
}
```

Output

```
Enter a number: 1.5  
Enter a number: 2.4  
Enter a number: -3.4  
Enter a number: 4.2  
Enter a number: 0  
Sum = 4.70
```

The do...while loop executes at least once i.e. the first iteration runs without checking the condition. The condition is checked only after the first iteration has been executed.

```
do {  
    printf("Enter a number: ");  
    scanf("%lf", &number);  
    sum += number;  
}  
while(number != 0.0);
```

So, if the first input is a non-zero number, that number is added to the sum variable and the loop continues to the next iteration. This process is repeated until the user enters 0.

But if the first input is 0, there will be no second iteration of the loop and sum becomes 0.0.

Outside the loop, we print the value of sum.